DESIGN AND ANALYSIS OF ALGORITHMS

LP-III Practical Viva | 100 Questions with Detailed Answers

How to use this question bank: Questions marked with ★ are high-priority questions that are especially likely in a practical viva. The remaining questions provide a mixture of basic definitions, conceptual questions, complexity analysis, practical implementation questions and applications. The answers use simple language and are generally 3–4 lines long.

Practical coverage: Fibonacci, Huffman Coding, Fractional Knapsack, 0/1 Knapsack and N-Queens. Theory coverage also includes the main DAA syllabus areas such as complexity classes, greedy methods, dynamic programming, backtracking, Branch and Bound, amortized analysis and distributed algorithms.

1. Algorithm Basics and Fundamentals

1. What is an algorithm? ★

Answer: An algorithm is a finite set of clear steps used to solve a problem. It accepts input, processes it and produces the required output. For example, a linear search algorithm checks each element until the required value is found.

2. What are the characteristics of a good algorithm? ★

Answer: It should have clearly defined inputs and outputs, unambiguous steps and a finite number of steps. Each operation should be simple and executable. It should also solve the given problem correctly and efficiently.

3. What is the difference between an algorithm and a program?

Answer: An algorithm is the logical step-by-step method for solving a problem. A program is the implementation of that method using a programming language such as Python. The same algorithm can be implemented in different languages.

4. What is algorithm analysis?

Answer: Algorithm analysis studies the resources required by an algorithm as the input size increases. The main resources are execution time and memory. It helps us compare different solutions before implementing them.

5. What is time complexity? ★

Answer: Time complexity describes how the number of operations grows with input size n. It is expressed using notations such as O(n), O(n log n) or $O(n^2)$. It does not directly represent time in seconds.

6. What is space complexity? ★

Answer: Space complexity measures the memory required by an algorithm as input size grows. It includes extra arrays, variables, data structures and recursion stack space. For example, an extra array of n elements requires O(n) space.

7. What is Big-O notation? ★

Answer: Big-O represents an asymptotic upper bound on the growth of an algorithm. For example, $O(n^2)$ means the growth is bounded by a constant multiple of n^2 for sufficiently large n. It is commonly used to express worst-case complexity.

8. Differentiate O, Ω and Θ notations. ★

Answer: O gives an upper bound, Ω gives a lower bound and Θ gives a tight bound. If an algorithm always processes n elements, its time is Θ(n). These notations describe growth rather than exact running time.

9. What are best, average and worst cases? ★

Answer: Best case is the minimum work for inputs of size n, while worst case is the maximum work. Average case is the expected work over a chosen distribution of inputs. For linear search, best case is O(1) and worst case is O(n).

10. Why do we ignore constants in asymptotic analysis?

Answer: For large input sizes, the fastest-growing term dominates the running time. For example, $4n^2 + 3n + 10$ has the same asymptotic growth as n^2. Therefore, we write $O(n^2)$ and ignore constants and lower-order terms.

11. What is an in-place algorithm?

Answer: An in-place algorithm uses very little extra memory, usually O(1), apart from the input. It modifies the input data directly instead of creating another large data structure. Selection sort is a common example.

12. What is recursion? ★

Answer: Recursion occurs when a function calls itself to solve a smaller version of the same problem. It must have a base case to stop the calls. Fibonacci and N-Queens are examples where recursion can be used.

13. What is a base case in recursion?

Answer: A base case is the condition that stops further recursive calls and returns a known result. Without it, recursion may continue until a stack or recursion-depth error occurs. In Fibonacci, F(0)=0 and F(1)=1 are base cases.

14. Differentiate iterative and recursive approaches.

Answer: An iterative approach uses loops such as for and while. A recursive approach solves the problem through function calls. Recursion can make some solutions easier to understand, but it uses additional call-stack memory.

15. How do you find the complexity of nested loops? ★

Answer: If two nested loops each run n times, the inner statement executes n×n times, giving $O(n^2)$. If the inner loop runs a fixed number of times, the total is O(n). Count how many times the main operation executes.

2. Complexity Classes and Problem Types

16. What is a polynomial-time algorithm?

Answer: An algorithm is polynomial time if its running time is bounded by a polynomial such as n, n^2 or n^3. Such algorithms are generally considered tractable for theoretical analysis. The actual performance still depends on constants and input size.

17. What is an exponential-time algorithm?

Answer: An exponential algorithm has running time such as O(2■) or O(3■). Its running time can grow very quickly as input size increases. Brute-force algorithms that examine all subsets often have exponential complexity.

18. What is class P? ★

Answer: P is the class of decision problems that can be solved by a deterministic algorithm in polynomial time. Examples include graph connectivity and shortest-path decision problems. These problems have efficient polynomial-time solutions.

19. What is class NP? ★

Answer: NP is the class of decision problems for which a proposed solution can be verified in polynomial time. For example, a proposed vertex cover can be checked efficiently by checking every edge. NP does not mean 'non-polynomial'.

20. What is an NP-hard problem?

Answer: A problem is NP-hard if every problem in NP can be reduced to it in polynomial time. It does not have to belong to NP or even be a decision problem. It is at least as hard as the hardest problems in NP.

21. What is an NP-complete problem? ★

Answer: A problem is NP-complete if it is both in NP and NP-hard. Its proposed solutions can be verified in polynomial time, and every NP problem can be reduced to it. Examples include 3-SAT and Vertex Cover decision problems.

22. Differentiate NP-hard and NP-complete.

Answer: Every NP-complete problem is NP-hard, but an NP-hard problem need not be NP-complete. NP-complete problems must also belong to NP, so their solutions can be verified in polynomial time. NP-hard problems may not satisfy this condition.

23. What is polynomial-time reduction?

Answer: It converts an instance of one problem into an instance of another problem in polynomial time while preserving the answer. It is used to compare problem difficulty. Reductions are important when proving NP-hardness and NP-completeness.

24. What is the Vertex Cover problem?

Answer: A vertex cover is a set of vertices such that every edge in the graph has at least one endpoint in the set. The decision problem asks whether a cover of size at most k exists. This decision problem is NP-complete.

25. What is the 3-SAT problem?

Answer: 3-SAT asks whether a Boolean formula with three literals in each clause can be made true by assigning values to its variables. It is a well-known NP-complete problem. It is often used to prove other problems NP-complete.

26. What is the Hamiltonian Cycle problem?

Answer: It asks whether a graph contains a cycle that visits every vertex exactly once and returns to the starting vertex. The decision problem is NP-complete. It is related to route-planning problems such as TSP.

3. Greedy Algorithms and Huffman Coding

27. What is a greedy algorithm? ★

Answer: A greedy algorithm makes the best local choice at each step and normally does not change earlier choices. It works when these local choices can be proven to lead to a global optimum. Examples include Huffman Coding and Fractional Knapsack.

28. What is the greedy-choice property?

Answer: It means an optimal solution can be obtained by making a locally best choice first. After that choice, the remaining problem can be solved in the same way. This property must be justified for the problem.

29. What is optimal substructure? ★

Answer: A problem has optimal substructure when an optimal solution contains optimal solutions to its smaller subproblems. This property is used in greedy algorithms and dynamic programming. It allows a large solution to be built from smaller solutions.

30. Why does a greedy algorithm not work for every problem?

Answer: A locally best choice may prevent the algorithm from finding the global optimum. For example, selecting the best profit-to-weight ratio does not always solve 0/1 Knapsack optimally. A correctness argument is required before using a greedy method.

31. What is Huffman Coding? ★

Answer: Huffman Coding is a lossless compression technique that assigns shorter binary codes to frequent symbols and longer codes to less frequent symbols. This reduces the total number of bits needed to store or transmit data. It is used in data-compression systems.

32. Why is Huffman Coding called a greedy algorithm? ★

Answer: At each step, it selects the two nodes with the smallest frequencies and combines them. This local choice is repeated until one tree remains. The resulting tree gives an optimal prefix code.

33. Explain the steps to construct a Huffman tree. ★

Answer: Create a leaf node for each symbol and insert the nodes into a min-priority queue. Remove the two nodes with the smallest frequencies and combine them under a new node with their summed frequency. Insert the new node and repeat until only one root remains.

34. What is a prefix code? Why is it useful? ★

Answer: A prefix code is a code in which no symbol's code is the prefix of another symbol's code. This makes decoding unambiguous because a codeword can be recognized when a leaf is reached. Huffman Coding produces prefix-free codes.

35. What are the time and space complexities of Huffman Coding? ★

Answer: With a min-heap and n distinct symbols, tree construction takes O(n log n) time. The tree has O(n) nodes, so its storage is O(n). Generating codes takes time proportional to the total length of the generated codes.

36. Why is a min-heap used in Huffman Coding? ★

Answer: A min-heap keeps the smallest frequency at its root. Huffman Coding repeatedly needs to remove the two smallest frequencies, and heap insertion/removal takes O(log n). This makes construction efficient.

37. How are Huffman codes generated?

Answer: Start at the root and assign 0 to a left edge and 1 to a right edge. The path from the root to a symbol's leaf becomes its binary code. The left/right convention can be reversed if encoding and decoding use the same tree.

38. How do you decode a Huffman bit string?

Answer: Start at the root and follow left for 0 and right for 1. When a leaf is reached, output its symbol and return to the root for the next bits. Continue until all bits are processed.

39. What happens if two symbols have equal frequencies?

Answer: Either of the tied minimum-frequency nodes can be selected first. The generated codes may differ, but the total encoded cost can still be optimal. A consistent tie-breaking rule makes program output reproducible.

40. Is Huffman Coding lossless or lossy?

Answer: Huffman Coding is lossless, meaning the original data can be reconstructed exactly after decoding. It changes the representation, not the information. This makes it suitable for data that must not be altered.

41. Where is Huffman Coding used in real life?

Answer: It is used in lossless data compression and as an entropy-coding technique in several file formats and compression systems. The goal is to reduce the number of bits used to represent data. It works particularly well when symbol frequencies are uneven.

4. Fractional Knapsack and Greedy Applications

42. What is the Fractional Knapsack problem? ★

Answer: We are given items with weights and profits and a bag with limited capacity. We may take a complete item or any fraction of it, with profit proportional to the fraction taken. The aim is to maximize total profit without exceeding capacity.

43. Explain the greedy approach for Fractional Knapsack. ★

Answer: Calculate the profit-to-weight ratio for every item and sort items in decreasing order of this ratio. Take items fully while capacity allows, then take a fraction of the next item if needed. Stop when the bag is full.

44. Why do we use profit-to-weight ratio?

Answer: The ratio tells us how much profit is gained per unit of weight. Selecting the highest ratio first gives the greatest profit for the available capacity. This strategy is optimal when fractions of items are allowed.

45. What are the time and space complexities of Fractional Knapsack? ★

Answer: Sorting n items takes O(n log n) time and selecting items takes O(n), so total time is O(n log n). The selection loop itself uses O(1) extra space. A separate sorted list may require O(n) space.

46. Can the same greedy method solve 0/1 Knapsack?

Answer: No. In 0/1 Knapsack, an item must be selected completely or rejected, so the best ratio may not lead to the best combination. Dynamic programming is commonly used to find the optimal solution.

47. What is Activity Selection?

Answer: It is a problem in which we select the maximum number of non-overlapping activities, given their start and finish times. The greedy strategy selects the activity that finishes earliest, then selects the next compatible activity. It is used in scheduling situations.

48. Why does Activity Selection choose the earliest finishing activity?

Answer: Choosing the activity that finishes earliest leaves the maximum remaining time for other activities. This choice can be shown to be part of an optimal solution. After sorting by finish time, the algorithm scans the activities once.

49. What is Job Sequencing with Deadlines?

Answer: Each job has a deadline and a profit, and usually takes one unit of time. A job earns profit only if it is completed by its deadline. The objective is to schedule jobs to maximize total profit.

50. Explain the greedy strategy for Job Sequencing.

Answer: Sort jobs by decreasing profit. For each job, place it in the latest free slot at or before its deadline. This keeps earlier slots available for jobs with tighter deadlines. A simple implementation may take O(n log n + nD), where D is the maximum deadline.

51. Give real-life applications of knapsack problems.

Answer: Knapsack models choosing projects under a limited budget, selecting files to fit on a storage device, or choosing equipment under a weight limit. Each item has a cost or weight and a benefit. The aim is to maximize benefit within a fixed limit.

5. Dynamic Programming, Fibonacci and 0/1 Knapsack

52. What is Dynamic Programming? ★

Answer: Dynamic Programming solves a problem by dividing it into smaller subproblems and storing their answers so they are not calculated repeatedly. It is useful when subproblems overlap and the problem has optimal substructure. Examples include Fibonacci and 0/1 Knapsack.

53. What are the two main properties of Dynamic Programming? ★

Answer: The first is overlapping subproblems, where the same smaller problems are solved repeatedly. The second is optimal substructure, where an optimal solution can be built from optimal solutions to subproblems. These properties make storing results useful.

54. Differentiate memoization and tabulation. ★

Answer: Memoization is top-down: it uses recursion and stores answers when subproblems are first solved. Tabulation is bottom-up: it fills a table using loops, starting from smaller cases. Both avoid repeated calculations.

55. What is the 0/1 Knapsack problem? ★

Answer: We are given n items with weights and values and a knapsack of capacity W. Each item must either be selected completely or not selected. The objective is to maximize total value without exceeding the capacity.

56. Explain the DP recurrence for 0/1 Knapsack. ★

Answer: Let K[i][w] be the best value using the first i items and capacity w. If item i is too heavy, use K[i–1][w]. Otherwise choose the maximum of excluding it and including it: max(K[i–1][w], value[i]+K[i–1][w–weight[i]]).

57. What are the time and space complexities of 0/1 Knapsack? ★

Answer: The standard table contains O(nW) states and each state takes constant time, so time is O(nW). The full two-dimensional table uses O(nW) space. A one-dimensional optimized version uses O(W) extra space.

58. Why is 0/1 Knapsack called pseudo-polynomial?

Answer: Its O(nW) running time depends on the numeric capacity W, not just the number of bits needed to represent W. The input needs only O(log W) bits to represent the capacity. Therefore, O(nW) is not necessarily polynomial in the encoded input length.

59. What is the Fibonacci sequence? ★

Answer: The Fibonacci sequence starts with 0 and 1, and each later term is the sum of the previous two terms. Its recurrence is F(0)=0, F(1)=1 and F(n)=F(n–1)+F(n–2). It is useful for explaining recursion and dynamic programming.

60. Compare recursive and iterative Fibonacci. ★

Answer: Naive recursion recalculates the same values many times, giving exponential time, commonly bounded by O(2■), and O(n) stack space. Iterative Fibonacci calculates each term once in O(n) time. If it stores only two previous values, its auxiliary space is O(1).

61. Why are base cases needed in Fibonacci?

Answer: The base cases F(0)=0 and F(1)=1 provide the first known values and stop recursion. Without them, the function would continue calling itself. They also define the sequence correctly.

62. How can you find selected items from the Knapsack DP table?

Answer: Start from K[n][W] and compare it with K[n–1][W]. If the value differs, item n was included, so subtract its weight and move to the previous row. If the values are equal, skip that item and continue.

63. What is Matrix Chain Multiplication?

Answer: It finds the best order to multiply a chain of matrices while minimizing the number of scalar multiplications. Matrix multiplication is associative, so different parenthesizations give the same result but may require different work. Dynamic Programming finds the minimum-cost order.

64. What is an Optimal Binary Search Tree?

Answer: An Optimal Binary Search Tree arranges keys using their search frequencies to minimize expected search cost. Frequently searched keys are generally placed closer to the root. Dynamic Programming evaluates possible roots and subtrees.

65. Differentiate Greedy and Dynamic Programming. ★

Answer: Greedy makes a locally best choice at each step and usually does not reconsider it. Dynamic Programming evaluates subproblem choices and stores their results to find an optimal combination. Fractional Knapsack uses greedy, while 0/1 Knapsack uses DP.

66. What are applications of Dynamic Programming?

Answer: DP is used in resource allocation, text processing, route planning and optimization. Common examples include 0/1 Knapsack, Fibonacci, Matrix Chain Multiplication and edit distance. It is especially useful when subproblems overlap.

6. Backtracking, Branch and Bound and N-Queens

67. What is Backtracking? ★

Answer: Backtracking builds a solution one decision at a time and checks whether the partial solution can still be valid. If a choice leads to a dead end, it is undone and another choice is tried. It is used in N-Queens, Sudoku and graph coloring.

68. What is a state-space tree?

Answer: A state-space tree represents all possible choices in a problem. Each node represents a partial or complete solution, and each branch represents a decision. Backtracking avoids exploring branches that cannot lead to a valid solution.

69. What is the N-Queens problem? ★

Answer: The task is to place N queens on an N×N chessboard so that no two queens attack each other. Therefore, no two queens can share a row, column or diagonal. It is a classic backtracking problem.

70. Explain the Backtracking approach for N-Queens. ★

Answer: Place one queen in each row and try each column. Before placing a queen, check whether its column and diagonals are free. If it is safe, move to the next row; if no position works, remove the previous queen and try another position.

71. How do you check whether two queens share a diagonal? ★

Answer: For positions (r1,c1) and (r2,c2), they share a diagonal when abs(r1−r2) equals abs(c1−c2). If one queen is placed in each row, row conflicts are already avoided. The column must also be checked.

72. What are the time and space complexities of N-Queens?

Answer: The search is exponential; a commonly quoted upper bound for row-by-row placement with pruning is O(N!), before including the cost of safety checks. A scan-based safety check may add O(N) work per candidate. The recursion and position array use O(N) auxiliary space; an explicit board uses $O(N^2)$.

73. What is pruning in Backtracking? ★

Answer: Pruning means stopping a branch as soon as it violates a constraint or cannot lead to a valid solution. In N-Queens, an unsafe queen placement is rejected immediately. This avoids exploring useless possibilities.

74. Differentiate Backtracking and brute force.

Answer: Brute force may generate every complete candidate and then test it. Backtracking checks partial candidates and stops exploring a branch as soon as it becomes invalid. It can reduce work, although worst-case time may still be exponential.

75. What is Branch and Bound?

Answer: Branch and Bound explores a state-space tree and calculates bounds on the best solution possible from each node. If a node cannot improve the best solution found so far, it is pruned. It is used for optimization problems such as TSP and Knapsack.

76. Differentiate Backtracking and Branch and Bound. ★

Answer: Backtracking usually prunes a branch when it cannot produce a feasible solution. Branch and Bound prunes a branch when its bound shows it cannot improve the current best objective value. Backtracking is common in constraint problems, while Branch and Bound is common in optimization.

77. What are FIFO, LIFO and Least-Cost Branch and Bound?

Answer: FIFO expands the oldest live node first using a queue. LIFO expands the newest live node first using a stack. Least-Cost expands the most promising node according to its bound or cost, usually using a priority queue.

78. What is the Sum of Subsets problem?

Answer: Given a set of numbers and a target sum, the task is to find subsets whose values add up to the target. Backtracking explores including or excluding each element. With positive values, a branch can be pruned when its partial sum exceeds the target.

79. What is Graph Coloring?

Answer: Graph Coloring assigns colors to vertices so that no two adjacent vertices have the same color. Backtracking tries colors for each vertex and rejects a color if a neighbour already has it. It is used in scheduling and register allocation.

80. Why is N-Queens not solved by a simple greedy choice?

Answer: A queen position may be safe now but block all possible placements in later rows. A greedy method may get stuck without checking alternatives. Backtracking reverses choices and explores other positions.

81. What is the base case in N-Queens recursion?

Answer: When the current row becomes N, all N queens have been placed safely. The program has found a complete solution and can print or store the board. This condition stops further recursive calls.

7. Complexity and Practical Coding Questions

82. What is the time complexity of linear search?

Answer: Linear search checks elements one by one until it finds the target or reaches the end. Its best case is O(1) when the first element matches, and its worst and average cases are O(n). It uses O(1) auxiliary space.

83. What is the time complexity of binary search?

Answer: Binary search repeatedly divides a sorted search range into half. Its time complexity is O(log n), and iterative binary search uses O(1) auxiliary space. It requires the input to be sorted.

84. What is the difference between O(n log n) and O(n^2)?

Answer: O(n log n) grows more slowly than O(n^2) for large n. Sorting algorithms such as merge sort commonly take O(n log n), while simple nested-loop sorting may take O(n^2). The difference becomes important for large datasets.

85. What is the time complexity of Python list append?

Answer: Appending to the end of a Python list is O(1) amortized. Occasionally, the list must allocate a larger memory block and copy elements, which can take O(n). Over many appends, the average cost remains constant.

86. Why is a min-heap useful in the Huffman practical?

Answer: A min-heap gives quick access to the smallest frequency node. Each insertion and removal takes O(log n), which is important because Huffman repeatedly combines the two smallest nodes. It gives O(n log n) construction time.

87. What do heapq.heappush() and heapq.heappop() do?

Answer: heappush() inserts an element while maintaining the heap property, and heappop() removes and returns the smallest element. Both take O(log n) time. They are useful for implementing Huffman's priority queue in Python.

88. What is the difference between == and = in Python?

Answer: = assigns a value to a variable, while == compares two values. The comparison returns True or False. Confusing them can cause errors in conditions and assignments.

89. What is a dry run of a program?

Answer: A dry run means tracing a program manually with a sample input. We follow the values of variables and the execution of loops or recursive calls. It helps explain the logic and identify mistakes without running the code.

90. What edge cases should you test in Fractional Knapsack?

Answer: Test zero capacity, capacity larger than total weight, one item and items with different ratios. Also check for invalid or zero weights because profit divided by zero is undefined. These tests verify that the program handles unusual inputs safely.

91. What edge cases should you test in 0/1 Knapsack?

Answer: Test zero capacity, no items, an item heavier than the capacity and a case where all items fit. Also test items with equal weights or values. These cases help verify table initialization and the include/exclude decisions.

92. How can N-Queens safety checking be made faster?

Answer: Instead of scanning all previous queens, maintain sets or boolean arrays for used columns and diagonals. For a queen at row r and column c, diagonals can be indexed by r−c+N−1 and r+c. These checks take O(1) average time.

93. What is the difference between returning and printing a value in Python?

Answer: return sends a result back to the calling function so it can be stored or reused. print only displays the value on the screen. A practical function should generally return a result when another part of the program needs it.

94. Why should we validate practical input?

Answer: Input validation checks that values are in the expected range and format before running the algorithm. For example, Fibonacci input should be a non-negative integer and knapsack weights should be valid. It prevents incorrect results and runtime errors.

95. How do you explain the time complexity of a recursive program?

Answer: Write the recurrence or count how many recursive calls are made at each level. Also consider the work done inside each call and the maximum recursion depth. For Fibonacci, repeated calls cause exponential time even though stack depth is only O(n).

96. Which practical algorithms from your list use Greedy, DP and Backtracking?

Answer: Huffman Coding and Fractional Knapsack use Greedy strategies. 0/1 Knapsack uses Dynamic Programming, while N-Queens uses Backtracking. Fibonacci can be solved using recursion or Dynamic Programming.

97. What is the difference between a min-heap and a max-heap?

Answer: In a min-heap, the smallest value is at the root, while in a max-heap, the largest value is at the root. Both are complete binary trees that maintain an ordering property. Huffman Coding uses a min-heap because it repeatedly needs the smallest frequencies.

98. Why can the same problem have more than one algorithm?

Answer: Different algorithms can solve the same problem using different strategies. One may be easier to implement, while another may use less time or memory. For example, Fibonacci can be solved using recursion, memoization or an iterative loop.

99. What is a time-space trade-off?

Answer: A time-space trade-off means using more memory to reduce execution time, or using less memory at the cost of extra computation. Memoized Fibonacci stores previous answers to avoid recalculation. The choice depends on available memory and performance needs.

100. How would you explain your practical if the examiner asks you to start from the main function?

Answer: First explain how the program accepts its input and validates it. Then describe which function performs the main algorithm and what data structures it uses. Finally, explain how the result is returned or printed and state the time and space complexity.